# Table of Contents


- Table of Contents
  - Introduction
  - Data Description
  - Key Challenges
    - Sparsity
    - Cold-Start
    - Bias/Variance
  - Baseline Models
    - Global Mean
    - Movie Mean
    - Movie-User Bias Model
  - Advanced Models
    - Movie-User-Time Model
    - SVD
    - KNN
  - Ensemble
  - Interpretation
    - Individual Models
    - Ensemble model
  - Results
  - Limitations
  - Conclusion
  - Extra Credit
  - Project Structure
    - Project contents
  - Additional Notes


## Introduction

In 2006, Netflix released over 100 million ratings from over 400,000 users on over 17,000 films. Whoever improved the RMSE of Netflix's "Cinematch" algorithm by 10% within 5 years received $1,000,000. Here, we revisit this contest by using a subset of this dataset, with a different test set than the original contest.

With the rise of social media, recommendation algorithms are more important than ever. Even children and non-technical people throw around the term "The Algorithm," showing its prescence in the current zeitgeist. Despite this challenge being annouced in 2006, I chose this project as I believed the skills I would learn from it would be especially relevant in the current landscape.

The goal of this project is to improve this RMSE by as much as possible using an ensemble of an SVD, K nearest neighbors, and baseline algorithm. This project was assigned during finals season to simulate working conditions; to see what I can produce when multiple projects are demanding my attention.

Though sources online differ, the RMSE (root mean square error) of the base "Cinematch" algorithm falls somewhere in the range of 0.9474 to 0.9525. With my limited resources, both time and compute,

surpassing this baseline I would consider a success (spoiler - RMSE achieved: 0.93409).

Though much of what I've done in this project is not required, I wanted to use this opportunity to demonstrate my computer science skills I've been teaching myself, as well as my math skills to some degree as I complete my math major after the end of this semester.

## Data Description

I was given two files: 'data.txt' and 'movieTitles.csv'. The file 'data.txt' is a plaintext file containing movie ids and rating information (the user id, a rating 1-5, and date rated for each review). It is over 27 million lines long, containing ratings on 5k movies from ~400k users, and the data is rather sparse as most users haven't rated most movies. Due to the size and formatting of the data, I created a special function in 'src/parsing.py' called "load_ratings()" that reads the file line by line and converts the results into a pandas dataframe. I then use another function to save the dataframe as a parquet in 'data/processed/ratings.parquet'. I did this to improve efficiency when repeatedly loading the data and to reduce the amount of storage the data took up.

The file 'movieTitles.csv' contains a table of each movie id, its release year, its corresponding movie title. It is a csv file that is 5000 lines long. Because of the commas in the movie titles, I wrote a function that manually parses each column and returns the data as a dataframe.

I then created a train-test split by using *all* ratings for a given movie in the training set, except for *one* randomly picked rating. That one rating (per movie) constitutes the test set. This was done with the "train_test" function in 'src/preprocessing.py'.

## Key Challenges

### Sparsity

Since most users haven't rated most movies, the data is very sparse, i.e. many user-movie combinations do not exist in the data. With sparse data, certain models aren't as well suited and overfitting becomes even more of a concern. My approach to this problem involved

- Using aggregation for the baseline models which reduces variance via pooling
- Residual modeling, in the user/movie/time bias model, capturing systematic over/underestimation trends common with sparse data
- SVD, designed to take large, sparse matrices and decompose them into smaller, denser matrices capturing important latent features

### Cold-Start

A problem with recommendation systems is the cold-start problem -- what do you do when there is little to no information on a user/movie? I handled this in my code by

- Using the global mean for movies with no reviews (if they exist)
- Using 0 user bias for new users
- Using the mean movie rating for pairs of new users and new movies

### Bias/Variance

To balance bias and variance in my data, I used

- Regularization terms that shrink the impact of movies with few ratings
- Filtering out users with few (<20) ratings for time bias calcualations

# Baseline Models

## Global Mean

The most simple baseline model is the global mean model. This model takes the average rating for all movies in the data and predicts this value no matter what. It is the best estimator when there is no other information available, but does not adapt to the movie or user.

## Movie Mean

This baseline model finds the average rating for each individual movie and, regardless of the user, predicts this value. This provides more flexibility than the global mean model as it adapts to differences between movies. However, it is does not account for user taste.

## Movie-User Bias Model

This model is modeled after the idea that

$$ \text{Actual Rating} \approx \text{Global Mean} + \text{Movie Bias} + \text{User Bias} $$

where movie bias is the term accounting for how much higher or lower rated the movie is than average and user bias accounts for whether the user has a tendency to give high ratings or low ratings. Additionally, there is a regularization constant that shrinks the movie and user biases toward 0, preventing movies and users with few ratings and extreme values from impacting the outcome too much.

# Advanced Models

## Movie-User-Time Model

This model extends the movie-user bias model by accounting for how user preferences change over time. The idea is users tastes and generosity with ratings may change as time passes. This model can be written as

$$ \text{Actual Rating} \approx \text{Global Mean} + \text{Movie Bias} + \text{User Bias} + \text{Time Effect} $$

The time effect, modeled as

$$ \text{Time Effect} = \alpha_{u} \cdot (t-\bar{t}_u) $$

where the $\alpha_u$ coefficient represents how a user's behavior changes over time and $\bar{t}_u$ is the average time of rating for that user. We subtract $\bar{t}_u$ from the actual time value to center it which allows us to properly capture whether the user became more or less generous over time. Otherwise, if a user had rated mostly when time was larger, then this trend would've likely been attributed to user bias instead. The regularization constant for movies and users is still there, along with a larger regularization constant for time, since time is a weaker signal and contains more noise. Additionally, the

time effect is modeled to predict the residuals to help capture the variance not explained by the previous model.

### SVD

I trained an SVD model with 50 factors and a reg_all value of 0.05 over 20 epochs using the surprise module. I chose to use an SVD model as it captures the interaction between movies and user taste. SVD models the user–item interaction matrix as the product of lower dimensional user and movie embeddings. Each rating is then approximated by the dot product of these latent factors. This allows SVD to learn underlying patterns in the data and then generalize better. In this project, the SVD model was the best single predictor of ratings.

### KNN

I trained a K nearest neighbors model where $k=40$ using surprise. I chose this model as I believed it would complement the SVD model in the ensemble later. SVD captures the overall latent structure while the KNN model caputures the local trends. Similar movies will tend to receive similar ratings, which is what this model is designed to capture.

## Ensemble

After being inspired by similar projects, I decided to create an ensemble of an SVD model, a KNN model, and the movie-user-time bias model. I chose these models because I theorized that SVD would capture the overall structure of the data, KNN would identify local patterns, and the movie-user-time bias model would incorporate temporal information. This gives use the model

$$ \text{Ensemble Prediction} = w_1 \cdot \text{SVD Prediction} + w_2 \cdot \text{KNN Prediction} + w_3 \cdot\text{Movie/User/Time Prediction} $$

Then, since the ensemble prediction is a linear equation, to find the appropriate weights for the ensemble, I used ridge regression with a basic grid search to find the best penalty coefficient. The idea behind this model is that, by combining diverse models that capture different elements of the signal, we can create a more robust and predictive model.

Indeed, we saw a drop in RMSE with this model, beating every other single predictor model. We found that the optimal combination of weights for SVD, KNN, and the movie/user/time model were $[0.6464 0.3105 0.0835]$. This shows that SVD had the greatest impact, with KNN still having a large impact, and the movie/user/time model having a smaller but noticeable impact.

## Interpretation

### Individual Models

This project experimented with a variety of models, from simple baselines to more complex models and finally an ensemble. The results show the strength of individual models and how combining complementary models can achieve results greater than the sum of its parts.

The global mean and movie mean models perform poorly since they don't account for user taste. Including user taste in the model brings the RMSE close to the original Netflix "Cinematch" algorithm's level. This shows that a lot of the variance in the ratings can be explained by individual rating patterns.

We found that the best non-ensemble model was SVD. This makes sense as, by learning latent representations of users and movies, it captures the underlying structure of the data, allowing it to generalize better than any other model. This result is in line with the knowledge that SVD is well suited for this kind of problem.

KNN performed the second best on this data set. This shows the strength of capturing local relationships. This model still managed to outperform the baseline models.

Finally, the movie/user/time bias model provided a slight improvement over the main baseline models. This shows that, while there is some signal in the temporal data, it is not especially strong or predictive.

### Ensemble model

The ensemble model achieved the best overall performance, reducing RMSE beyond any individual model. By using ridge regression, we learned optimal weights directly from the data and minimized this ensemble's performance.

The learned weights placed the greatest emphasis on the SVD model, then KNN, then the user/movie/time bias model. However, the fact that they all contribute to a meaningful degree shows how they capture a different signal. The idea behind this ensemble is

- SVD captures global latent structure
- KNN captures local relationships
- User/movie/time bias models provides a baseline estimate and incorporates temporal information

Ensembling allows us to stack the strenghts of each model while helping to minimize the weaknesses of each individual model by having the others make up for them.

## Results

| Model | RMSE |
|---|---|
| Global Mean | 1.2832 |
| Movie Mean | 1.1002 |
| Bias Model | 0.9688 |
| Time Bias | 0.9656 |
| KNN | 0.9464 |
| SVD | 0.9393 |
| Ensemble | 0.9341 |

## Limitations

The data used in this project is fairly limited. Ideally, there would be information on genre, language, and other metadata that could be incorporated to make better predictions. Additionally, I was rather limited by computing power, with my models taking a total of 1hr 20min to train. This made it not feasible to implement cross validation, though if I could've, I would've. Because of this, I was forced into using small subsets of the data to tune hyperparameters.

## Conclusion

Our findings show that SVD is the strongest individual model on this data set, followed by K nearest neighbors, and then our baseline movie/user/time bias model. However, these models do not all capture the same signal. By combining different models that capture different signals, we can create a stronger, more robust, and more predictive model, leading to the ensemble method being the most effective.

## Extra Credit

I used the latent vectors from my SVD and projected them into 2D using UMAP. We can see that movies from the same series are grouped together such as Back to the Future parts II and III. Horror movies also are grouped together, with Scream, Texas Chainsaw Massacre, and Tremors all nearby each other. We can also see a very "Dad" kind of movie taste, with War Games, The Godfather, Part II, and The Wire all clustered together. Even though the model was trained on ratings, we can see how it learned to capture patterns in user taste.

## Project Structure

### Project contents

- README.md
- pyproject.toml
    - Setup for this project
    - Run [pip install -e .] to install packages
- .vscode/
    - settings.json
        - Settings for VSCode
- .gitignore
    - Ignore data and configurations
- data/
    - Files to big to upload to GitHub
    - raw/
        - data.txt
            - Contains movie rating information
        - movieTitles.csv
            - Contains movie titles and IDs
    - processed/
        - ratings.parquet
            - A parquet containing the parsed data.txt file
- notebooks/
    - exploration.ipynb
        - Used for exploring data before implementing
- src/
    - __init__.py
        - Treat this directory as a package
    - config.py
        - N-number and paths to data
    - parsing.py
        - Getting the data into the dataframes and parquet
    - preprocessing.py
        - Getting the data into a more workable state
    - baseline.py
        - Train baseline models
    - svd.py
        - Train svd model
    - knn.py
        - Train K nearest neighbors model
    - evaluation.py
        - Evaluate the models
- img/
    - Images
- models/
    - svd_model.pkl
    - knn_model.pkl
        - Files containing the trained models (since they take a long time to train)
        - Files too big to upload to GitHub

- predictions/
    - bias_model_preds.parquet
    - global_mean_preds.parquet
    - movie_mean_preds.parquet
    - knn_preds.parquet
    - svd_preds.parquet
    - time_bias_model_preds.parquet
        - Contains the predictions made by each model
- main.py
    - Main python file

## Additional Notes

Some of the inspiration for the models used in this project came from various sources about what worked in the original competition. Additionally, AI (ChatGPT 5.3) was used for the following tasks:

- Debugging code
- Syntax help with implementing the models
- Troubleshooting package managment and dependency problems
- Checking the math/logic of the baseline models
  Additonally, I took inspiration from some of the suggestions the AI made during the debugging process, such as how to optimize code and memory.